Lab 6: Servo & Ultrasonic Rangefinding

Engineer: Stanley Etienne / J00974053
Course: ECEL-360
Date: 02-25-26

Objective

The objective of this lab was to mount an ultrasonic rangefinder on a servo motor and control the direction of the sensor using a joystick module. The joystick's analog input was used to position the servo, while the joystick push button was used to trigger a distance measurement. When the button was pressed, the ultrasonic sensor measured the distance to an object and displayed the result in centimeters on the Serial Monitor. This lab demonstrated joystick calibration, button debouncing, and ultrasonic distance measurement using the ESP32.

Materials

- ESP32 Development Board
- SG90 Servo Motor
- HC-SR04 Ultrasonic Sensor
- Joystick Module
- External 5V Power Supply (for the servo)
- Breadboard
- Jumper wires
- Two resistors for ECHO voltage divider (optional)
- USB cable
- Arduino IDE

Background and Theory

Servo Motor Control

When controlled by a Pulse Width Modulated (PWM) signal, a servo motor turns to a certain angle. The ESP32 makes PWM signals that tell the servos to move to different positions. In

this lab, the joystick's analog value was mapped to a range of 0° to 180°, which made the servo turn based on where the joystick was.

Joystick Operation

The joystick module sends out an analog signal that changes based on where it is on the horizontal axis. The microcontroller on the ESP32 has a 12-bit analog-to-digital converter (ADC), so analog readings can be anywhere from 0 to 4095.

Ultrasonic Distance Measurement

The HC-SR04 ultrasonic sensor sends out an ultrasonic sound pulse and measures how long it takes for the echo to come back to figure out how far away something is. The sensor sends out a short pulse that makes an ultrasonic burst. The sensor raises the ECHO pin for the time it takes for the sound wave to go to the object and back.

Button Debouncing

When you press a mechanical push button, it often makes several quick changes because of the bounce of the physical contact. To stop a single press from sending multiple sensor readings, software debouncing was used. This makes sure that each button press only triggers one valid reading.

Circuit Layout:

Wiring Configuration:

Component	ESP32 Pin	Description
Servo Signal	GPIO23	Servo control using PWM
Ultrasonic TRIG	GPIO27	Trigger pin for ultrasonic sensor
Ultrasonic ECHO	GPIO26	Echo pin through voltage divider
Joystick VRx	GPIO34	Analog input for joystick direction
Joystick SW	GPIO13	Push button input with INPUT_PULLUP

| Servo V+ | 5V | External power supply |

Procedure

1. Mounted the ultrasonic sensor onto the servo motor so that the sensor rotates with the servo.
2. Connected the joystick module to the ESP32 with the X-axis connected to GPIO34.
3. Connected the joystick push button to GPIO13 using the internal pull-up resistor.
4. Connected the ultrasonic sensor TRIG pin to GPIO27 and the ECHO pin to GPIO26 through a voltage divider.
5. Connected the servo signal wire to GPIO23 and powered the servo with an external 5V supply.
6. Programmed the ESP32 to read the analog joystick value.
7. Mapped the joystick values to a servo angle range between 0° and 180°.
8. Implemented joystick calibration to determine minimum and maximum analog values.
9. Implemented software debouncing for the joystick push button.
10. Programmed the ultrasonic sensor to take a distance measurement when the button was pressed.
11. Displayed the measured distance in centimeters on the Serial Monitor.

Code:

```
#include <ESP32Servo.h>

// ---------------- Pin Definitions ----------------
#define SERVO_PIN   23
#define TRIG_PIN    27
#define ECHO_PIN    26
```

```
#define X_AXIS_PIN  34
#define JOY_BTN_PIN 13

// ---------------- Ultrasonic Settings ----------------
#define MAX_DISTANCE_CM 500
float timeOut = MAX_DISTANCE_CM * 60;
int soundVelocity = 340;

// ---------------- Servo Object ----------------
Servo myServo;

// ---------------- Joystick Calibration ----------------
int joyMin = 4095;
int joyMax = 0;

// ---------------- Debounce ----------------
bool lastBtnState = HIGH;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 200;

// ---------------- Ultrasonic Function ----------------
float getSonarCM() {
  unsigned long pingTime;
  float distance;

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  pingTime = pulseIn(ECHO_PIN, HIGH, timeOut);

  if (pingTime == 0) return -1;

  distance = (float)pingTime * soundVelocity / 2 / 10000;
  return distance;
}

// ---------------- Joystick Calibration ----------------
void calibrateJoystick(unsigned long ms = 2000) {
  unsigned long start = millis();
  while (millis() - start < ms) {
    int v = analogRead(X_AXIS_PIN);
    if (v < joyMin) joyMin = v;
```

```
    if (v > joyMax) joyMax = v;
    delay(5);
  }

  if (joyMax - joyMin < 200) {
    joyMin = 0;
    joyMax = 4095;
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(JOY_BTN_PIN, INPUT_PULLUP);

  myServo.attach(SERVO_PIN);

  Serial.println("Move joystick left/right for calibration...");
  calibrateJoystick(2000);
  Serial.printf("Calibrated: Min=%d Max=%d\n", joyMin, joyMax);
  Serial.println("Press joystick button to take distance reading.\n");
}

void loop() {

  // ---- Servo control ----
  int joystickValue = analogRead(X_AXIS_PIN);
  int angle = map(joystickValue, joyMin, joyMax, 0, 180);
  angle = constrain(angle, 0, 180);

  myServo.write(angle);
  delay(10);

  // ---- Button press detection (debounced) ----
  bool currentBtnState = digitalRead(JOY_BTN_PIN);

  if (lastBtnState == HIGH && currentBtnState == LOW) {
    unsigned long now = millis();
    if (now - lastDebounceTime > debounceDelay) {

      lastDebounceTime = now;

      float distance = getSonarCM();
```

```
      Serial.printf("Servo Angle: %d° | Distance: ", angle);

      if (distance < 0)
        Serial.println("Out of range");
      else
        Serial.printf("%.2f cm\n", distance);
    }
  }

  lastBtnState = currentBtnState;
}
```

Results

The system operated as expected during testing. The joystick successfully controlled the position of the servo motor, allowing the ultrasonic sensor to rotate smoothly across its range of motion. When the joystick was moved left or right, the servo rotated proportionally to the joystick's analog input value.

Pressing the joystick button successfully triggered the ultrasonic sensor to take a distance measurement. The measured distance was displayed in centimeters on the Serial Monitor. The software debouncing mechanism prevented multiple readings from being triggered by a single button press.

The joystick calibration ensured that the full motion of the joystick corresponded to the complete servo rotation range. This resulted in accurate positioning of the ultrasonic sensor during scanning.

Overall, the system demonstrated stable operation and reliable distance measurements during testing.

Conclusion

This lab successfully demonstrated the integration of multiple input and output devices using the ESP32 microcontroller. The joystick was used as an analog input device to control the orientation of a servo motor, while the joystick button served as a digital trigger for ultrasonic distance measurement.

The lab reinforced several key embedded systems concepts, including analog input processing, PWM-based servo control, button debouncing, and ultrasonic time-of-flight distance measurement. By combining these components, a simple directional rangefinding system was created that allowed the user to aim the sensor and take distance measurements interactively.

This experiment provided practical experience with sensor integration and control systems, which are fundamental skills in embedded system design.